CHAPITRE II

REPRESENTATION DES DONNEES

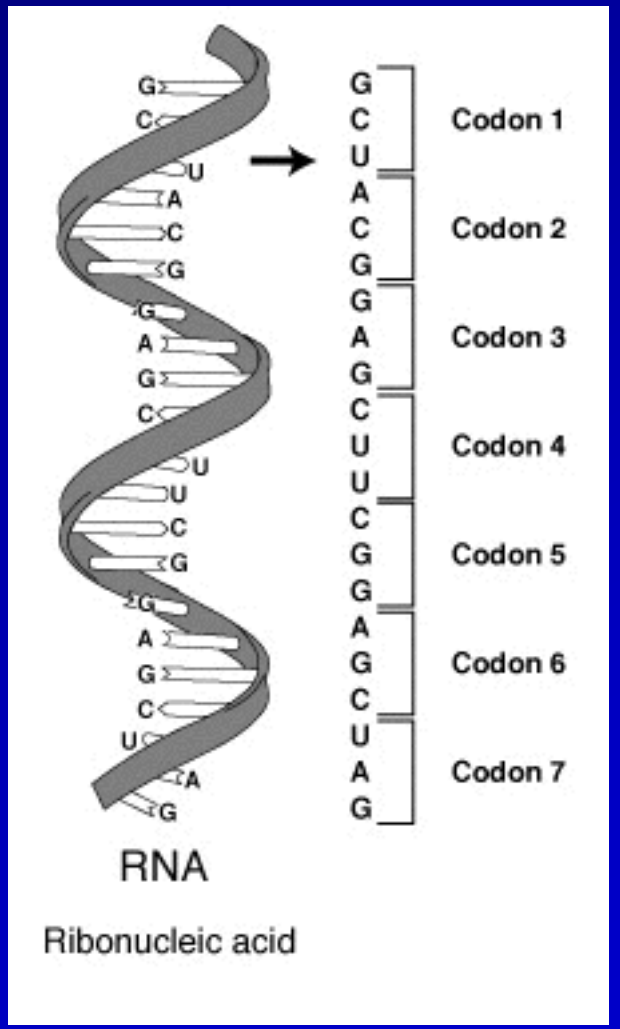
I - INTRODUCTION:

L'informatique est la science du traitement de l'information. Apparue au milieu du 20ème siècle, elle a connu une évolution extrêmement rapide.

A la motivation initiale qui était de faciliter et d'accélérer le calcul, se sont ajoutées de nombreuses fonctionnalités, comme l'automatisation, le contrôle et la commande de processus, la communication ou le partage de l'information.

II – CODAGE DE L'INFORMATION

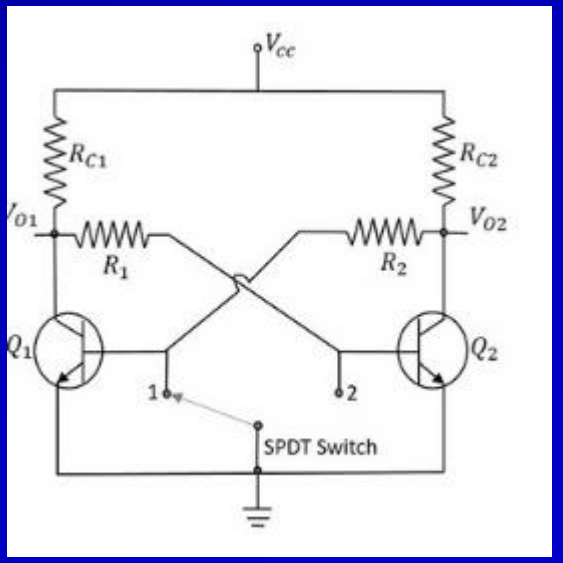
Pour traiter l'information, il faut la représenter sous une forme manipulable par le système de traitement.
Le codage d'une information consiste à établir une correspondance entre la représentation externe de l'information (exemple le caractère A ou le nombre 36), et sa représentation interne dans la machine.



1 – Codage et Opérations Binaires

Les informations traitées par un microprocesseur sont de différents types (nombres, instructions, images, vidéo, etc...) mais elles sont toujours représentées sous un format binaire. On utilise la représentation binaire car elle est simple, facile à réaliser techniquement à l'aide de bistables (système à deux états réalisés à l'aide de transistors).





En binaire, une information élémentaire est appelé bit et ne peut prendre que deux valeurs différentes : 0 ou 1.
Une information plus complexe sera codée sur plusieurs bits. On appelle cet ensemble un mot.
Un mot de 8 bits est appelé un octet.

a) Représentation d'un nombre entier en binaire :

Les nombres sont exprimés par des chiffres pouvant prendre deux valeurs 0 ou 1. A chaque chiffre est affecté un poids exprimé en puissance de 2.
Ex : $(101)_2 = 1x\ 2^2 + 0x2^1 + 1x2^0 = (5)_{10}$

b) Représentation d'un nombre entier en hexadécimal

Lorsqu'une donnée est représentée sur plus de 4 bits, on préfère souvent l'exprimer en hexadécimal.
Les nombres sont exprimés par des chiffres et des lettres pouvant prendre 16 valeurs : 0 1 2 3 4 5 6 7 8 9 A B C D E F
A chaque chiffre est affecté un poids exprimé en puissance de 16.

Ex : $(9A)_{16} = 9x16^1 + Ax16^0 = 9x16 + 10x1 = (154)_{10}$

c) Utilisation d'une base quelconque - changement de base

Un nombre N en base B s'écrit :

$N_B = a_n a_{n-1} a_{n-2} \ldots a_1 a_0$ avec $0 \leq a_i \leq B-1$

Exemples :

base 2 (binaire) : 01101110010_2 $0 \leq a_i \leq 1$

base 8 (octal) : $12456764706 20_8$ $0 \leq a_i \leq 7$

base 10 (décimal) : 974682056_{10} $0 \leq a_i \leq 9$

base 16 (hexadécimal) : $3A5E8F_{16}$ $0 \leq a_i \leq F$

chiffres hexadécimaux : 0 1 2 3 4 5 6 7 8 9 A B C D E F

Pour passer d'une base à une autre il est toujours possible de passer par la base 10
Pour passer du binaire à une base de type 2^n, il suffit de regrouper les bits par n de la droite vers la gauche afin de retrouver chaque chiffre en base 2^n.

Exemples : 10110111_2 à représenter en base 8 et 16

en octal ($8 = 2^3$) : $010\ \ 110\ \ 111_{\ 2} = 267_8$
en hexadécimal ($16 = 2^4$) : $1011\ \ 0111_{\ 2} = B7_{16}$

Exercices :

1) On donne le nombre 32745 en base 8.
   Comment s'écrit-il en base 16 ?

a) passer par l'intermédiaire de la base 10 ;

b) passer par l'intermédiaire de la base 2.

2) Un nombre s'écrit 753 en base 8.
   Comment s'écrit-il en base 16 ?

a) passer par l'intermédiaire de la base 10 :

b) passer par l'intermédiaire de la base 2.

3) Effectuer : $1110101_2 + 11000011_2$

n bits permettent de représenter 2^n valeurs différentes, donc les nombres entiers positifs de 0 à 2^n-1.
Exemple : sur 8 bits on peut représenter $2^8 = 256$ valeurs différentes donc les entiers de 0 à 255
(0000 0000 à 1111 1111).
EXEMPLES :
4 bits / 6 bits / 8 bits
Généralement :
Avec n bits on représente les nombres entiers de -2^{n-1} à $+2^{n-1} - 1$.

EXERCICES : APPLICATION AU RESEAU

Une adresse IP est codée sur 32 bits (soit 4 octets).

Afin de simplifier la lecture et l'écriture d'adresses IP pour les humains, on a choisi d'écrire les adresses avec la notation en décimal pointée. Cette dernière sépare les 4 octets sous forme de 4 chiffres décimaux allant de 0 à 255.

Cela donne par exemple : 192.168.0.1

EXERCICE 1 : Convertir les adresses IP suivantes en binaires :

192.168.5.1 84.22.161.192 213.41.120.195

NOTION DE CLASSES D'ADRESSES IP :

En septembre 1981, est publié le RFC790 qui entérine la création de trois classes d'adresses A, B et C.

La création des classes D et E n'est intervenue que plus tard. Ce système de classes a perduré jusqu'à la fin des années 1990. Pourquoi étudier un système obsolète aujourd'hui ?

Parce que son importance fut telle sur les développements d'Internet qu'inévitablement, on rencontre de façon régulière des références aux classes. Les nouvelles versions du cursus CCNA nomment l'adressage avec classes, « adressage hérité ».

LES CLASSES D'ADRESSES IP



EXERCICE 2 :

Déterminer les plages d’adresses IP correspondant aux classes A, B et C.

MASQUE DE SOUS-RÉSEAU :

Pour définir les parties réseau et hôte d'une adresse, on utilise une configuration de 32 bits appelée « masque de sous-réseau ». Le masque de sous-réseau s'exprime dans le même format décimal pointé que celui de l'adresse IPv4. Le masque de sous-réseau est créé en plaçant le nombre binaire 1 dans chaque position de bit qui représente la partie réseau et en plaçant le nombre binaire 0 dans chaque position de bit qui représente la partie hôte.

EXERCICE 3 :

Pour chaque adresse IP, trouver :

- ✓ La classe d'adresse standard
- ✓ Le masque de sous-réseau

Adresse IP	192.168.4.27	172.16.1.32	10.10.32.14	83.206.12.14	167.23.45.12
Classe					
Masque					

NB : masque pour réseau IP non découpé

d) Représentation des nombres binaires signés

La solution la plus simple consiste à réserver un bit pour ce signe. Les autres bits représentant une valeur absolue du nombre. La convention retenue impose de mettre le premier bit (qu'on appelle aussi bit de poids fort) à 0 pour repérer un nombre positif et à 1 pour un nombre négatif.

On parle alors de donnée signée quand on utilise cette convention.

Toutefois, une telle représentation des nombres signés entraînerait un traitement spécial du signe et des circuits électroniques.

Cet inconvénient est résolu par une autre forme de représentation des nombres négatifs dite représentation en complément à 2 (C2) ou encore représentation sous forme complémentée.

☞ Pour un nombre entier positif, sa représentation en complément à 2, c'est sa représentation en base 2.

☞ Pour un nombre entier négatif, on obtient sa représentation en complément à 2 en 3 étapes :

1- calculer la représentation binaire de sa valeur absolue

2- complémenter tous les bits (0→1, 1→0)

3- ajouter 1 (additionner 1 au nombre obtenu)

Exercices : -23 sur 8 bits

-100 sur 8 bits

NB :

- La représentation C2 n ’a de sens que si on précise qu’on l ’utilise pour ce nombre. [-23 en C2 vaut 233 en binaire].
- Lorsque des opérations sont effectuées sur des nombres codés en complément à 2, ceux-ci doivent impérativement être codés sur le même nombre de bits.
- Le bit de poids fort d'un nombre négatif est à 1, le bit de poids fort d'un nombre positif est à 0.

Exercices : sur 4 bits

4	0100	4	0100
6	0110	-2	1110

Comment détecter un débordement ?

si les nombres sont de signes différents, pas de Problème ;

S’ils sont de même signe, le signe du résultat doit être le même, sinon il y a débordement.

Exercice :

On considère ces opérations écrites en base 10 :

a) – 61 – 44

b) – 61 – 72

c) 99 – 35

d) 99 + 35

On dispose d'une machine travaillant sur des nombres binaires de longueur 8 (8 bits).

Faire manuellement ce que l'additionneur de la machine ferait automatiquement, et donner les résultats obtenus en binaire. En cas d'erreur, indiquer pourquoi

NB :

1 - Complément à deux : addition, signes opposés

Le résultat est toujours représentable

S'il n' y a pas de retenue, on lit le résultat directement. S'il y a une retenue, on la néglige

2 - Addition de deux nombres de même signe

L'addition de deux nombres de même signe peut donner lieu à un dépassement de capacité !

- Cas de deux entiers de signe positif :

On a un dépassement de capacité quand la retenue est distincte du dernier bit de report (i.e. celui sur le bit de signe).

- Cas de deux entiers de signe négatif :

On a toujours une retenue, que l'on oublie.
On a un dépassement de capacité quand la retenue est distincte du dernier bit de report (i.e. celui sur le bit de signe).

e) Représentation des nombres réels

Soit par exemple le nombre fractionnaire 12,346

En décimal il s'écrit :

$12,346 = 1x10^{1} + 2x10^{0} + 3\ x10^{-1} + 4\ x10^{-2} + 6\ x10^{-3}$

Pour la partie entière, on procède par divisions

comme pour un entier.

On multiplie la partie fractionnaire par la base en conservant la partie entière chaque fois et on répète l'opération sur la partie fractionnaire du produit jusqu'a ce qu'elle soit nulle (ou que la précision voulue soit atteinte)

Exemple : conversion de $(54,25)_{10}$ en base 2

- Partie entière : $(54)_{10} = (110110)_2$ par divisions.
- Partie fractionnaire :

$0,25 \times 2 = 0,50 \Rightarrow a^{-1} = 0$

$0,50 \times 2 = 1,00 \Rightarrow a^{-2} = 1$

$0,00 \times 2 = 0,00 \Rightarrow a^{-3} = 0$

$(54,25)_{10} = (110110,010)_2$

Exercices d'application :

$(73,55)_{10} = (\quad)_2$

$(312,3)_4 = (\quad)_8$

Additionner 7,25 et 2,5 en binaire

Convertir les nombres suivants de leur base vers les 3 autres :

10	2	8	16
362,3125			
	10111101,101		
		326,5	
			F3C7,A

CODAGE EN VIRGULE FIXE ET VIRGULE FLOTTANTE :

Soit le nombre décimal 3,25

CODAGE EN VIRGULE FIXE :

Etant donné une base b, un nombre x est représenté en format virgule fixe, par :

$X = a_{n-1}a_{n-2}\ldots a_1a_0,a_{-1}a_{-2}\ldots a_{-p}$ (b)

a_{n-1} est le chiffre de poids fort (MSB)

a_{-p} est le chiffre de poids faible (LSB)

n est le nombre de chiffre avant la virgule

p est le nombre de chiffre après la virgule

CODAGE EN VIRGULE FLOTTANTE :

On a adopté une notation scientifique appelée format à virgule flottante : $x = \pm M \cdot 2^{E}$

La notation flottante comprend trois (3) composantes :

- le signe
- le significande (appelé aussi mantisse)
- l'exposant de la puissance

Exemple :

$(3,25)_{10} = 11,01_{(2)}$ (en virgule fixe)

$= 1,101 . 2^{1}$

$= 110,1 . 2^{-1}$

La notation est dite flottante car elle consiste à faire glisser les nombres significatifs à droite ou à gauche

☞Pb : différentes manières de représenter E et M

d'où la nécessité de Normalisation

Standard IEEE 754 (1985 / 2008)

NOTATION :

Les nombres réels sont normalisés sous la forme

- ☞ $1,\ldots \times 2^n$ puis codés sur 32bits.
- ☞ le 1er bit code le signe (0 pour + et 1 pour -)
- ☞ les 8 bits suivants codent l'exposant **auquel on ajoute 127** (permet de coder des exposants négatifs sans C2)
- ☞ les 23 derniers bits codent la partie fractionnaire du nombre (après la virgule) mais si le 24ième bit est 1 alors on arrondit à la valeur supérieure (mantisse + 1)

- Simple précision sur 32 bits :

  1 bit de signe de la mantisse

  8 bits pour l'exposant

  23 bits pour la mantisse

SM	E	23

1 bit 8 bits 23 bits

- Double précision sur 64 bits :

  1 bit de signe de la mantisse

  11 bits pour l'exposant

SM	E	52 bits

Exemple :

5,3→ 101,0100110011001100110011001100110011001…

Normalisation :

1,0101001100110011001100110011001100110011001.. * 2^2

signe : 0

Exposant : 2 + 127 = 129 soit sur 8 bits 1000 0001

mantisse sur 23 bits : 01010011001100110011001

mais le 24ième bit est 1 donc la mantisse est

01010011001100110011010

codage final :

0100 0000 1010 1001 1001 1001 1001 1010

de manière plus lisible en hexadécimal :

40 A9 99 9A

Remarques :

l'exposant 0000 0000 signifie que le nombre est dénormalisé

l'exposant 1111 1111 signifie que ce n'est pas un nombre (NaN)

En simple précision, les exposants sont donc compris entre -126 et 127

Exercice :

Convertir le nombre décimal 8,625 en virgule flottante suivant la norme IEEE 754 :

8 = 1000

0,625 x2 =1,25 $a^{-1} = 1$

0,25 x2 = 0,50 $a^{-2} = 0$

0,50 x2 = 1,00 $a^{-3} = 1$

8,625 = 1000,101 = 1000,101 x 2^0

= 0,1000101x2^4 (Normalisation)

= 1,000101x2^3 (Pseudo-normalisation IEEE 754)

Signe	Exposant biaisé	Pseudo-mantisse

EXERCICE

1°) Soit un nombre stocké en mémoire :

1011 1101 0100 0000 0000 0000 0000 0000

Donner ce nombre en décimal

2°) Soient les 2 nombres codés suivant la norme IEEE 754 et représentés en hexadécimal : **3EE00000** et **3D800000**. Représenter les 2 nombres sous forme décimale.

Calculer en la somme et donnez le résultat sous forme IEEE 754 et sous forme décimale.

e) Décimal Codé Binaire et Code de Gray

Le Décimal Codé Binaire ou Binary Coded Decimal (BCD), permet de coder des nombres entiers positifs.
En BCD, on code chaque chiffre du nombre décimal sur 4 bits et on concatène ces demi-octets.
Exemple : 4236 devient 0100 0010 0011 0110 en BCD

Le code de Gray, ou code Gray ou code binaire réfléchi est un codage binaire permettant de ne modifier qu'un seul bit à la fois quand un nombre est augmenté d'une unité. Cette propriété est importante pour plusieurs applications.

Codage décimal	Codage binaire	Codage Gray
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100

Une méthode consiste à calculer le OU EXCLUSIF entre le binaire de départ et ce même binaire décalé d'un rang à droite

Exercice :

$(5\ 7\ 6)_{10} = (\quad)_{\mathbf{BCD}}$

$(9\ 9\ )_{10} = (\quad)_{\mathbf{BCD}}$

$(1000\ 0011\ 0110)_{BCD} = (\quad)_{\mathbf{10}}$

Exercice :

Représenter 10 ; 15 et 34 en code Gray

2 - Représentation des caractères

Les caractères sont des données non numériques : il n’y a pas de sens à additionner ou multiplier deux caractères. Par contre, il est souvent utile de comparer deux caractères, par exemple pour les trier dans l’ordre alphabétique.
Le codage des caractères est fait par une table de correspondance indiquant la configuration binaire représentant chaque caractère (ASCII).

DEUX STANDARDS :

- Table ASCII Etendu

  8 bits pour représenter 256 caractères (0 à 255)

  Code les caractères accentués : à, é,…etc.

  Compatible avec ASCII

- Code Unicode (mis au point en 1991)

  16 bits pour représenter 65 536 caractères

  Compatible avec ASCII

  Code la plupart des alphabets : Arabe, Chinois, …

  On a défini environ 50 000 caractères à ce jour

Table ASCII standard (codes de caractères de 0 à 127)

000		(nul)	016	▶	(dle)	032	sp	048	0	064	@	080	P	096	`	112	p
001	☺	(soh)	017	◀	(dc1)	033	!	049	1	065	A	081	Q	097	a	113	q
002	☻	(stx)	018	↕	(dc2)	034	"	050	2	066	B	082	R	098	b	114	r
003	♥	(etx)	019	‼	(dc3)	035	#	051	3	067	C	083	S	099	c	115	s
004	♦	(eot)	020	¶	(dc4)	036	$	052	4	068	D	084	T	100	d	116	t
005	♣	(enq)	021	§	(nak)	037	%	053	5	069	E	085	U	101	e	117	u
006	♠	(ack)	022	▬	(syn)	038	&	054	6	070	F	086	V	102	f	118	v
007	•	(bel)	023	↨	(etb)	039	'	055	7	071	G	087	W	103	g	119	w
008	◘	(bs)	024	↑	(can)	040	(	056	8	072	H	088	X	104	h	120	x
009		(tab)	025	↓	(em)	041	)	057	9	073	I	089	Y	105	i	121	y
010		(lf)	026		(eof)	042	*	058	:	074	J	090	Z	106	j	122	z
011	♂	(vt)	027	←	(esc)	043	+	059	;	075	K	091	[	107	k	123	{
012	♀	(np)	028	∟	(fs)	044	,	060	<	076	L	092	\	108	l	124	|
013		(cr)	029	↔	(gs)	045	-	061	=	077	M	093	]	109	m	125	}
014	♫	(so)	030	▲	(rs)	046	.	062	>	078	N	094	^	110	n	126	~
015	☼	(si)	031	▼	(us)	047	/	063	?	079	O	095	_	111	o	127	⌂

CODE ISO-8859 -1 (Iso-latin-1)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STH	ETH	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	CD2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL
8	€	□	‚	ƒ	„	…	†	‡	ˆ	‰	Š	‹	Œ	□	Ž	□
9	□	'	'	"	"	•	–	—	˜	™	š	›	œ	□	ž	Ÿ
A		¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬	-	®	¯
B	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
C	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
D	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
E	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
F	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

Exercices :

1. En utilisant le code ASCII, écrire votre nom et votre prénom sans oublier de mettre les initiales en majuscule.

2. En utilisant le code ASCII, décoder la phrase suivante : 76 101 32 98 105 110 97 105 114 101 32 99 96 101 115 116 32 102 97 99 105 108 101 46